Easy Search

Plataforma de búsqueda inteligente de establecimientos, productos y servicios

Propuesta técnica y plan de trabajo

PostgreSQL + pgvector · FastAPI · React · IA generativa

Entrega del MVP: 25 de noviembre de 2026

1. Análisis general

Easy Search es, en esencia, un sistema de **recuperación aumentada por generación (RAG) con componente geográfico**. Permite que cualquier persona o empresa escriba en lenguaje natural lo que necesita (por ejemplo, *"Quiero comer hamburguesas en Bucaramanga, pero tengo máximo 25.000 pesos"*) y obtenga los establecimientos más adecuados, con una explicación de por qué se recomiendan.

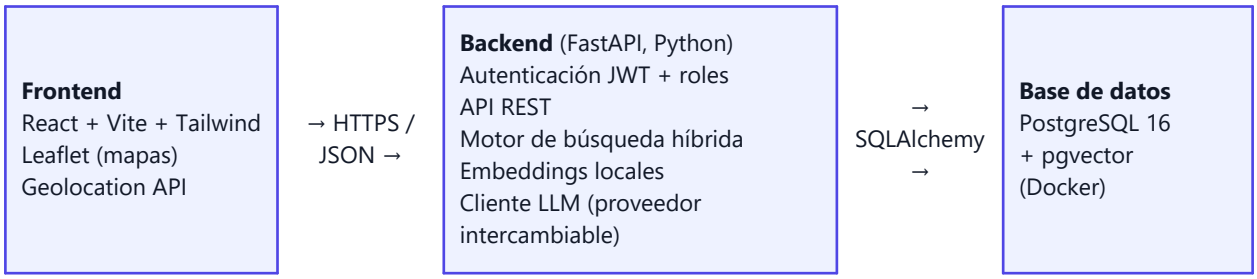
Tiene tres problemas técnicos centrales:

- **Entender la consulta:** convertir el texto libre en datos estructurados: qué busca, dónde, presupuesto, condiciones y preferencias.
- **Recuperar:** combinar similitud semántica (pgvector) con filtros SQL exactos: zona, distancia, precio, horario, calificación y categoría.
- **Explicar:** la IA redacta recomendaciones usando **únicamente** los datos recuperados de la base de datos, sin inventar precios, horarios ni reseñas.

El mayor riesgo del proyecto no es la IA sino **los datos**: un buscador semántico sin establecimientos reales y bien descritos no sirve. Por eso la base inicial se construirá con datos de **OpenStreetMap** y se enriquecerá con la información que aporten los propietarios. El resto de funcionalidades (favoritos, historial, reseñas, paneles) son operaciones CRUD con roles y permisos, de bajo riesgo técnico.

2. Arquitectura tecnológica

Se propone un **monolito modular**: un backend, un frontend y una base de datos. Sin microservicios ni colas externas, porque no aportan valor en el alcance inicial y complicarían el desarrollo.



3. Tecnologías seleccionadas y justificación

Capa	Tecnología	Justificación
Base de datos	PostgreSQL 16 + pgvector (imagen Docker *pgvector/pgvector:pg16*)	Requisito obligatorio. Docker evita compilar pgvector en Windows y garantiza el mismo entorno en cualquier equipo.
Backend	Python + FastAPI	El desarrollador ya conoce Python y el ecosistema de IA/embeddings vive en Python. FastAPI valida datos con Pydantic y genera documentación Swagger automáticamente.
ORM y migraciones	SQLAlchemy 2 + Alembic + pgvector-python	Consultas parametrizadas (protección contra SQL Injection) y migraciones versionadas del esquema.
Embeddings	intfloat/multilingual-e5-small (sentence-transformers, 384 dimensiones, ejecución local)	Muy buen rendimiento en español, gratuito y sin depender de una API externa. Es reemplazable si se requiere.

Capa	Tecnología	Justificación
LLM	Proveedor intercambiable: Google Gemini o Anthropic Claude (decisión pendiente)	Interpreta la consulta (JSON estructurado), redacta recomendaciones y apoya la moderación. Se implementará detrás de una interfaz común para cambiar de proveedor sin tocar el resto del sistema.
Autenticación	JWT + contraseñas con Argon2	Estándar, seguro y sin dependencia de terceros. Roles: usuario, propietario y administrador.
Frontend	React + Vite + Tailwind CSS + React Router	Basado en JavaScript, conocido por el desarrollador. Tailwind permite un diseño propio y responsive.
Mapas	Leaflet + teselas de OpenStreetMap	Gratuito y sin API key.
Datos iniciales	OpenStreetMap (Overpass API), licencia ODbL	Datos reales de nombres, direcciones, teléfonos, sitios web y horarios; se cita la atribución requerida.
Geolocalización	navigator.geolocation + tabla propia de municipios y barrios	La ubicación precisa solo con permiso del usuario; las ubicaciones escritas se resuelven sin APIs externas.
Distancias	Fórmula de Haversine en SQL con prefiltro por rectángulo	Suficiente para el volumen del MVP; evita la complejidad adicional de PostGIS.
Pruebas	pytest (backend) + pruebas manuales guiadas (frontend)	

Descartados: Next.js (SSR innecesario), microservicios, Elasticsearch (pgvector y el full-text de PostgreSQL son suficientes) y Google Places (costoso y sus términos prohíben almacenar sus datos).

4. Búsqueda semántica con PostgreSQL y pgvector

Modelo de datos del núcleo

- **establishments:** nombre, descripción, coordenadas, municipio, rango de precios, calificación promedio, zona horaria y fecha de última verificación de horarios.
- **offerings:** productos y servicios con precio opcional (p. ej. “Hamburguesa doble – $22.000”, “Despinche de llantas de buseta”).
- **categories:** jerarquía flexible; un establecimiento puede tener varias.
- **embeddings:** (tipo de entidad, id, hash del contenido, vector(384)): uno por establecimiento y uno por cada producto o servicio.
- **opening_hours** y **special_hours:** varias franjas por día (horarios divididos, cruce de medianoche) y excepciones por fecha (festivos, cambios de horario).

Indexación

- Al crear o modificar un establecimiento o producto se construye un **texto canónico** con su información relevante.
- Se calcula un hash del texto: **solo si cambió** se regenera el embedding, en segundo plano.
- Índice **HNSW** con operador de distancia coseno (*vector_cosine_ops*).

Flujo de una consulta

Paso	Descripción
1. Interpretación	El LLM convierte la consulta en JSON validado (necesidad, ubicación, presupuesto, abierto ahora, preferencias). Si el LLM falla, un intérprete por reglas actúa como respaldo.
2. Ubicación	"Cerca de mí" usa las coordenadas del navegador (si el usuario las autorizó); una ciudad o barrio escritos se resuelven con la tabla de municipios. Nunca se asume una ubicación.
3. Embedding	Se genera el vector de la necesidad interpretada con el mismo modelo de la indexación.
4. SQL híbrido	Una sola consulta combina filtros duros (radio o municipio, presupuesto, calificación, horario), ranking semántico (*embedding* <=> *vector_consulta*) y ranking léxico (full-text en español). Los rankings se fusionan con Reciprocal Rank Fusion y se ajustan por calificación y distancia.
5. Recomendación	Los mejores resultados se envían al LLM, que redacta una recomendación por establecimiento usando solo esos datos. Si falta información, lo indica explícitamente.

Esto es búsqueda vectorial real: pgvector es el motor central, no se reemplaza por búsquedas LIKE. Para que los filtros no vacíen los resultados del índice HNSW se usará *hnsw.iterative_scan* (pgvector 0.8 o superior).

Horarios de atención

El estado se calcula con la hora actual **en la zona horaria del establecimiento**, revisando primero las excepciones por fecha y luego el horario semanal. El resultado es uno de tres estados: **abierto**, **cerrado** o **sin información**. Nunca se afirma que un negocio está abierto sin datos que lo respalden.

5. Estructura del proyecto

```
Easy-Seach/
├── docker-compose.yml        PostgreSQL + pgvector
├── .env.example              variables de entorno (sin claves reales)
├── README.md
├── docs/                     arquitectura, avance, manual de pruebas
├── backend/
│   ├── app/
│   │   ├── main.py
│   │   ├── core/             configuración, seguridad, dependencias de auth
│   │   ├── models/           modelos SQLAlchemy
│   │   ├── schemas/          esquemas Pydantic
│   │   ├── api/routes/       auth, search, establishments, reviews,
│   │   │                     favorites, history, owner, admin
│   │   ├── services/         search, embeddings, llm, hours, moderation
│   │   └── db/
│   ├── alembic/              migraciones
│   ├── scripts/              seed, importación OSM, reindexación
│   └── tests/
└── frontend/
    └── src/
        ├── pages/            Home, Results, Detail, Login, Profile,
        │                     Favorites, History, Owner, Admin
        ├── components/
        ├── services/         cliente de la API
        └── context/          autenticación, ubicación
```

6. Plan de trabajo por fases

Las fases siguen el orden de dependencias: **0 → 1 → 2 → 3 → (4 y 5 en paralelo) → 6 → 7 → 8 → 9**. No se avanza a una fase nueva sin cumplir el criterio de terminación de la anterior.

Fase 0 — Entorno de desarrollo	
Objetivo	Que backend, frontend y base de datos arranquen en el equipo del desarrollador.
Actividades	• Configurar docker-compose con PostgreSQL + pgvector. • Esqueleto de FastAPI y de React (Vite + Tailwind). • Archivos .env.example y .gitignore.
Tecnologías	Docker, PostgreSQL, pgvector, FastAPI, React, Vite.
Entregables	Repositorio con estructura base y README de instalación.
Criterio de terminación	La extensión vector está activa, GET /health responde y la página de React abre en el navegador.

Fase 1 — Modelo de datos y autenticación	
Objetivo	Esquema completo de la base de datos y gestión de usuarios con roles.
Actividades	• Modelos SQLAlchemy y migraciones Alembic. • Roles: usuario, propietario, administrador. • Registro e inicio de sesión con JWT y Argon2. • Tabla de municipios y barrios con coordenadas. • Importación de establecimientos reales desde OpenStreetMap.
Tecnologías	SQLAlchemy, Alembic, Pydantic, JWT, Argon2, Overpass API.
Entregables	Esquema de base de datos, endpoints de autenticación, script de importación.
Criterio de terminación	Un usuario puede registrarse e iniciar sesión, un endpoint de administrador rechaza a usuarios normales y la base contiene establecimientos reales.

Fase 2 — Establecimientos, horarios y embeddings	
Objetivo	Catálogo completo con vectores semánticos.
Actividades	• CRUD de establecimientos, productos, servicios y categorías. • Horarios por día, divididos, excepciones y zona horaria; estado abierto / cerrado / sin información. • Pipeline de embeddings con índice HNSW. • Regeneración automática de embeddings al cambiar la información.
Tecnologías	pgvector, sentence-transformers, FastAPI.
Entregables	API del catálogo, servicio de embeddings, pruebas de horarios.
Criterio de terminación	Editar una descripción regenera su embedding y pasan las pruebas de horarios (horario dividido, cruce de medianoche, festivo).

Fase 3 — Motor de búsqueda inteligente	
Objetivo	Construir el núcleo del producto: búsqueda semántica combinada con filtros.
Actividades	• Interpretación de la consulta con el LLM y respaldo por reglas. • Resolución de ubicación: ciudad escrita, cercanía o solicitud al usuario. • Consulta SQL híbrida: vectorial + full-text + filtros, con fusión de rankings. • Endpoint /search.
Tecnologías	pgvector, full-text de PostgreSQL, LLM, Pydantic.
Entregables	Búsqueda funcional y pruebas con los cuatro ejemplos de la propuesta.
Criterio de terminación	Los cuatro ejemplos devuelven resultados coherentes y los filtros modifican los resultados.

Fase 4 — Frontend de búsqueda	
Objetivo	Construir la experiencia principal del usuario.
Actividades	• Sistema de diseño propio. • Página de inicio con el buscador como protagonista. • Resultados con filtros y mapa. • Detalle del establecimiento. • Permiso de geolocalización. • Diseño responsive para móviles.
Tecnologías	React, Tailwind, Leaflet, Geolocation API.
Entregables	Páginas de Inicio, Resultados y Detalle.
Criterio de terminación	El flujo buscar → filtrar → ver detalle → ver en el mapa funciona desde un celular.

Fase 5 — Recomendaciones IA y cuenta de usuario	
Objetivo	Aportar el valor de la IA y la personalización.
Actividades	• Recomendaciones basadas solo en datos recuperados, con validación anti-invención. • Login, registro y perfil. • Favoritos. • Historial de búsquedas: consultar, repetir y eliminar.
Tecnologías	LLM, React, FastAPI.
Entregables	Recomendaciones en resultados y páginas de cuenta.
Criterio de terminación	En diez consultas de prueba ninguna recomendación menciona datos inexistentes en la base.

Fase 6 — Reseñas y panel comercial	
Objetivo	Habilitar la comunidad y la gestión por parte de los propietarios.
Actividades	• Reseñas y calificaciones: crear, editar y eliminar las propias. • Moderación: reglas, clasificación con LLM y cola de pendientes. • Límite de frecuencia; una reseña por usuario y establecimiento. • Solicitud de propiedad y verificación. • Panel del propietario: perfil, productos, horarios.
Tecnologías	FastAPI, React, LLM.
Entregables	Sistema de reseñas moderado y panel comercial.
Criterio de terminación	Nadie puede editar reseñas ajenas y un propietario solo puede editar su establecimiento.

Fase 7 — Panel administrativo	
Objetivo	Dar control total de la plataforma a los administradores.
Actividades	• Gestión de usuarios, establecimientos y categorías. • Moderación de reseñas. • Aprobación de solicitudes de propietarios y verificación de perfiles. • Consulta de búsquedas realizadas. • Manejo de información incorrecta o duplicada. • Estadísticas generales.
Tecnologías	FastAPI, React.
Entregables	Panel administrativo.
Criterio de terminación	Todas las acciones administrativas funcionan y están protegidas por rol.

Fase 8 — Calidad y seguridad	
Objetivo	Asegurar la robustez del sistema.
Actividades	• Pruebas automatizadas del backend. • Revisión de seguridad: autorización, validación de entradas, CORS, límite de peticiones, privacidad de ubicación. • Manejo de errores. • Pulido del diseño responsive.
Tecnologías	pytest.
Entregables	Suite de pruebas y lista de verificación de seguridad.
Criterio de terminación	Todas las pruebas pasan y la lista de seguridad está completa.

Fase 9 — Documentación y cierre	
Objetivo	Dejar un proyecto entregable y reproducible.
Actividades	• Documentación de arquitectura, instalación, variables de entorno y API (Swagger). • Manual de usuario. • Guion de demostración. • Corrección de errores finales.
Tecnologías	—
Entregables	Documentación completa y demostración.
Criterio de terminación	El proyecto se instala desde cero siguiendo únicamente el README.

7. Entregables finales

- Repositorio en GitHub con el código del backend, el frontend y las migraciones.
- Base de datos PostgreSQL + pgvector reproducible con *docker compose up*.
- Buscador semántico funcional con filtros, ubicación y recomendaciones de IA.
- Funciones de usuario: favoritos, historial y reseñas moderadas.
- Panel comercial y panel administrativo con roles y permisos.
- Pruebas automatizadas del backend: autenticación, horarios, búsqueda y permisos.
- Documentación: arquitectura, instalación, variables de entorno, API y manual de pruebas.
- Datos reales de Bucaramanga y Piedecuesta importados desde OpenStreetMap.

8. Riesgos técnicos y limitaciones

Riesgo	Impacto	Mitigación
Datos incompletos (precios, fotos, reseñas)	Alto	Importación desde OpenStreetMap, curación manual de negocios clave y enriquecimiento por propietarios.
El LLM inventa información	Alto	Prompt restringido a datos recuperados, salida estructurada y validación de precios y horarios citados.
Proveedor de LLM aún no definido	Medio	Interfaz común con adaptadores para Gemini y Claude; el resto del sistema no depende del proveedor.
Costo o caída de la API del LLM	Medio	Intérprete por reglas como respaldo; la búsqueda vectorial funciona sin LLM (embeddings locales).
Horarios desactualizados	Medio	Estado “sin información” y fecha de última verificación visible.
Descarga inicial del modelo de embeddings (~470 MB)	Bajo	Se descarga una sola vez y queda en caché.
Precisión del GPS según dispositivo	Bajo	Ajuste manual de la ubicación; las coordenadas no se guardan en el historial.
Reseñas fraudulentas sofisticadas	Medio	Reglas, moderación con LLM, una reseña por usuario y establecimiento, cola de revisión manual.

Riesgo	Impacto	Mitigación
Plazo ajustado para el alcance completo	Alto	Priorizar la búsqueda; recortar primero estadísticas avanzadas y pulido visual.

9. Funcionalidades para versiones posteriores

Ninguna funcionalidad se elimina de la visión; las siguientes quedan preparadas en la arquitectura pero no se implementan completamente en el MVP:

- **Disponibilidad en tiempo real** (stock o cupos): requiere integración con cada negocio; el MVP muestra horarios.
- **Despliegue en producción** (dominio, HTTPS, servidor), si no es requisito de la entrega.
- **Almacenamiento de fotos en la nube**: el MVP las guarda en disco local.
- **Detección automática de duplicados**: el MVP sugiere posibles duplicados y el administrador los fusiona.
- **Detección avanzada de fraude** en reseñas (patrones entre cuentas, IP o dispositivo).
- **Inicio de sesión con Google, notificaciones, reservas y pedidos.**
- **Integración real con WhatsApp**: el MVP usa enlaces directos wa.me.
- **Personalización basada en el historial** del usuario.

10. Decisiones acordadas

Tema	Decisión
Fuente de datos	OpenStreetMap, complementado con información manual y de propietarios.
Proveedor de LLM	Pendiente (Gemini o Claude). Se implementa una interfaz intercambiable.
Infraestructura local	Docker Desktop disponible en el equipo de desarrollo.
Fecha de entrega	25 de noviembre de 2026.
Repositorio	github.com/SantiagovillamizarM/Easy-Seach